www.tntextbooks.in

To solve a problem recursively, the solver reduces the problem to sub-problems, and calls another instance of the solver, known as sub-solver, to solve the sub-problem. The input size to a sub-problem is smaller than the input size to the original problem. When the solver calls a sub-solver, it is known as recursive call. The magic of recursion allows the solver to assume that the sub-solver (recursive call) outputs the solution to the sub-problem. Then, from the solution to the sub-problem, the solver constructs the solution to the given problem.

As the sub-solvers go on reducing the problem into sub-problems of smaller sizes, eventually the sub-problem becomes small enough to be solved directly, without recursion. Therefore, a recursive solver has two cases:

1. **Base case:** The problem size is small enough to be solved directly. Output the solution. There must be at least one base case.

2. **Recursion step:** The problem size is not small enough. Deconstruct the problem into a sub-problem, strictly smaller in size than the given problem. Call a sub-solver to solve the sub-problem. Assume that the sub-solver outputs the solution to the sub-problem. Construct the solution to the given problem.

This outline of recursive problem solving technique is shown below.

```
solver (input)
    if input is small enough
        construct solution
    else
        find sub_problems of reduced input
        solutions to sub_problems =
        solver  for each sub_problem
        construct solution to the problem from
            solutions to the sub_problems
```

Whenever we solve a problem using recursion, we have to ensure these two cases: In the recursion step, the size of the input to the recursive call is strictly smaller than the size of the given input, and there is at least one base case.

8.4.3 Recursion — Examples

Example 8.11. The recursive algorithm for length of a sequence can be written as

```
length (s)
-- inputs : s
-- outputs : length of s
    if s has one customer -- base case
        1
    else
        1 + length(tail(s)) -- recursion step
```

Example 8.12. Design a recursive algorithm to compute a^n. We constructed an iterative algorithm to compute a^n in